

Migración a React de la Plataforma Web E-Commerce LevelUp

Nombre: Vicente Rodríguez / Sebastián Valderrama / Carlos
Pinilla

Asignatura: Full Stack II
Evaluación Parcial N°2

Índice

1. Introducción y motivos de la migración.....	3
Justificación de la migración.....	3
2. Tecnología utilizada.....	4
2.1. Versión HTML.....	4
2.2. Versión React.....	4
2.3. Herramientas de pruebas.....	5
3. Del sitio HTML al sitio React.....	6
4. Estructura del proyecto en React.....	7
4.1. Componentes reutilizables.....	7
4.2. Páginas del sitio.....	8
4.3. Navegación entre páginas.....	8
5. Propiedades y estado (React).....	9
5.1. Propiedades (props).....	9
5.2. Estado (state).....	9
6. Validaciones que se mantuvieron.....	11
6.1. Buscador de productos.....	11
6.2. Formulario de contacto.....	11
7. Pruebas unitarias (Karma y Jasmine).....	14
7.1. Configuración del entorno.....	14
7.2. Casos de prueba.....	15
8. Pendientes y mejoras futuras.....	16
9. Control de versiones con GitHub.....	17
9.1. El repositorio del proyecto.....	17
9.2. Commits.....	18
9.3. Trabajo en equipo.....	19
9.4. Beneficios para este proyecto.....	19
10. Conclusión.....	20

1. Introducción y motivos de la migración

LevelUp es una tienda de productos tecnológicos que reúne un catálogo, información de la empresa, datos de una sucursal y un formulario de contacto. La primera versión se desarrolló con HTML, CSS, Bootstrap y JavaScript. La migración conserva estas secciones y reorganiza su implementación mediante componentes React.

Este informe explica las decisiones de la migración, las tecnologías utilizadas y el funcionamiento del código actual. Los diagramas representan el flujo de datos y las decisiones de la búsqueda, el *contacto* y las pruebas.

Justificación de la migración

En HTML, la navegación y el pie de página se repetían en distintos archivos. Al convertirlos en CustomNav y Footer, los cambios de enlaces o autoría se realizan en un único lugar. Hero, Catálogo y las páginas informativas conservan responsabilidades separadas, lo que facilita localizar una modificación.

La búsqueda original modifica directamente la visibilidad de las tarjetas mediante selectores del DOM. En React, App almacena la consulta y Catálogo calcula los productos que deben renderizarse. El formulario también conserva sus reglas de validación, pero administra valores, errores y confirmación mediante estado.

React Router permite cambiar el contenido central según la ruta. Bootstrap y los estilos por componente mantienen la presentación visual. La migración responde al desarrollo académico del proyecto y mejora la organización del código; no se ha realizado una comparación que permita afirmar una mejora de rendimiento frente al sitio HTML.

2. Tecnología utilizada

El proyecto tiene dos versiones. Cada una usa un conjunto distinto de herramientas.

2.1. Versión HTML

- ❖ **HTML5:** arma la estructura de cada página.
- ❖ **CSS3:** define los colores y la apariencia, en el archivo `span.css`.
- ❖ **Bootstrap:** entrega la grilla y los componentes que adaptan el sitio a distintos tamaños de pantalla.
- ❖ **JavaScript:** el archivo `funciones.js` contiene el buscador de productos y la validación del formulario de contacto.

2.2. Versión React

- ❖ **React 19:** construye el sitio con componentes reutilizables. Se usan los paquetes `react` y `react-dom`.
- ❖ **react-router-dom 7:** permite cambiar de página sin recargar el sitio.
- ❖ **Bootstrap 5 y react-bootstrap:** conservan el diseño de la versión HTML. `react-bootstrap` ofrece el menú, la barra de navegación y la grilla como componentes de React.
- ❖ **Create React App (react-scripts 5.0.1):** permite iniciar el sitio con `npm start` y generar la versión final con `npm run build`.
- ❖ **Node.js y npm:** instalan las librerías del proyecto y ejecutan los comandos.
- ❖ **Google Maps:** el mapa de la página Sucursal está incrustado desde este servicio.
- ❖ **GitHub:** guarda el código de las dos versiones.

2.3. Herramientas de pruebas

- ❖ **Karma:** ejecuta las pruebas en el navegador Chrome y muestra el resultado en la consola.
- ❖ **webpack y babel-loader:** traducen el código de React para que el navegador lo entienda. style-loader y css-loader cargan los estilos.
- ❖ **Jasmine:** define los casos de prueba y verifica los resultados.

3. Del sitio HTML al sitio React

Cada página HTML se convirtió en un componente de React. Lo que se repetía en todas las páginas, como el menú y el pie de página, pasó a ser un único componente compartido. La Figura 1 muestra hacia dónde se movió cada pieza.

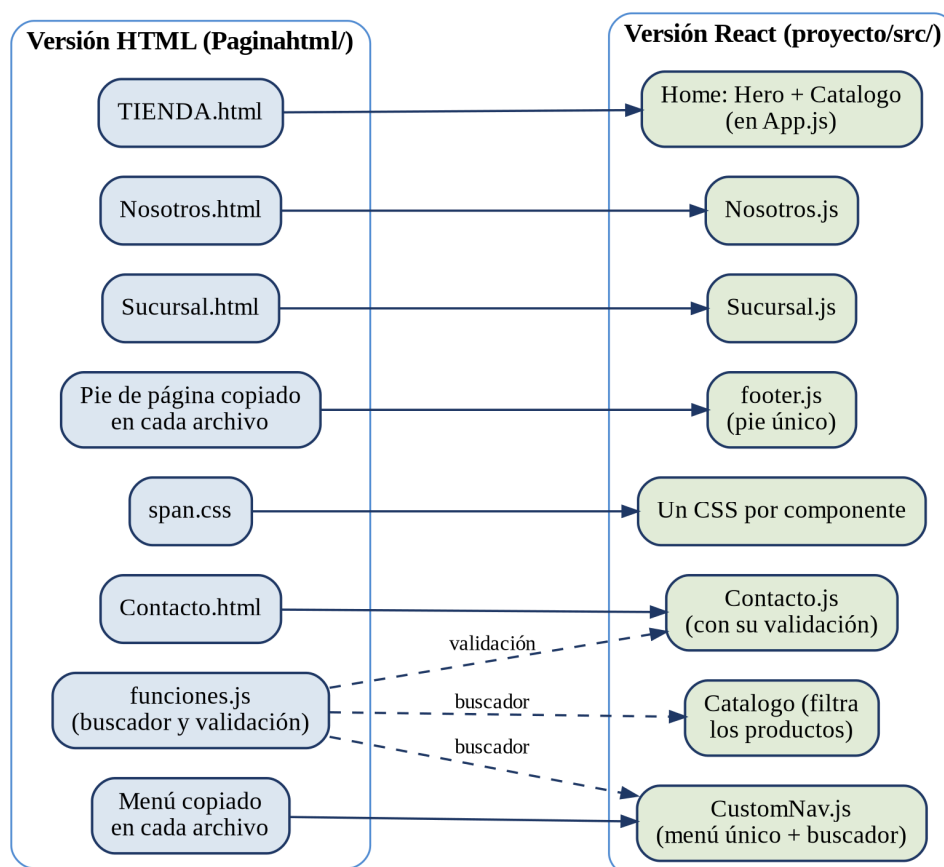


Figura 1. De la versión HTML a la versión React.

El archivo **funciones.js** se dividió en dos. El buscador pasó a CustomNav, que recibe lo que escribe el cliente, y a Catalogo, que filtra los productos. La validación del formulario pasó a Contacto.js. Los estilos de span.css quedaron repartidos en un archivo CSS por componente. El sitio mantiene sus cuatro páginas (Inicio, Nosotros, Sucursal y Contacto), con mucho menos código repetido.

4. Estructura del proyecto en React

El código está en la carpeta **proyecto/src/** y se ordena en dos grupos: **components/**, con las piezas reutilizables, y **paginas/**, con las pantallas completas. La Figura 2 muestra cómo se conectan.

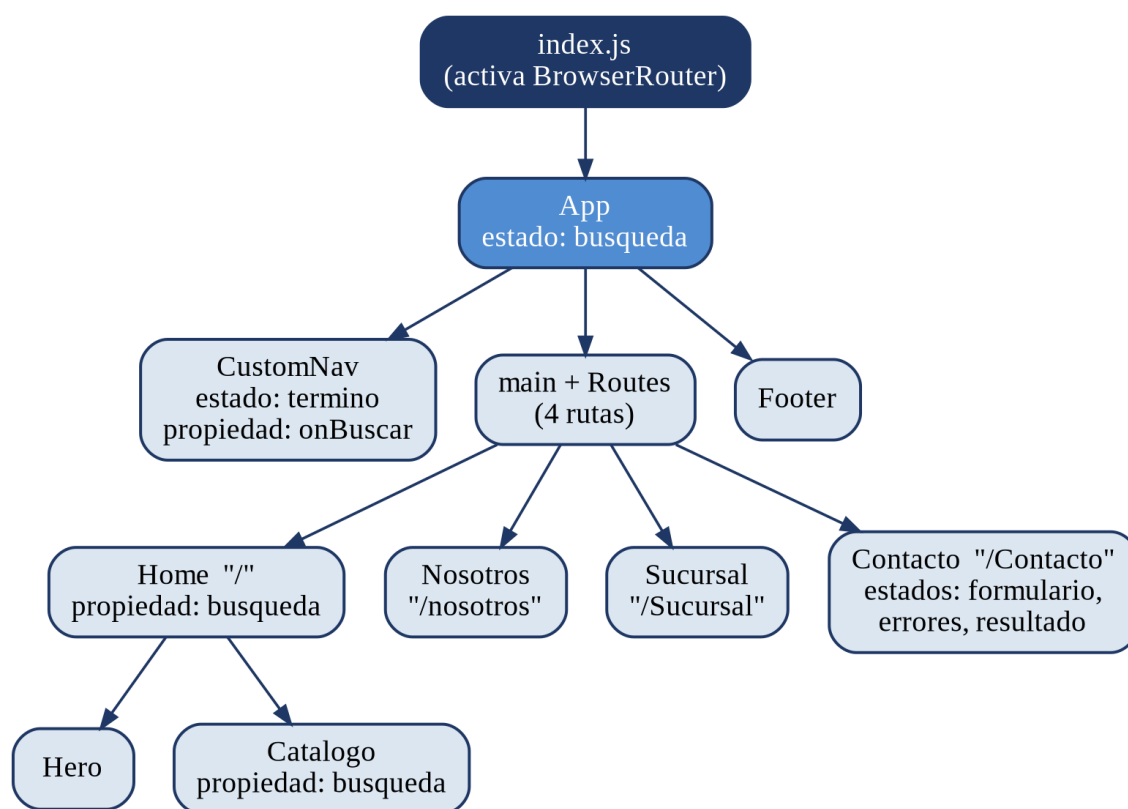


Figura 2. Componentes de la aplicación, sus rutas y dónde vive cada estado.

4.1. Componentes reutilizables

- ❖ **CustomNav:** barra de navegación superior con el logo y los enlaces Home, Nosotros y Sucursal. Un menú desplegable “Más” da acceso a Contacto. A la derecha tiene el buscador de productos. En pantallas pequeñas se pliega en un botón de tres líneas (menú “hamburguesa”).
- ❖ **Hero:** bloque de bienvenida con el título “Tienda Online”, un mensaje para el cliente y el botón “Explorar Productos”.

- ❖ **Catalogo:** muestra los productos en tarjetas. Los datos de los productos están en una lista dentro del componente y React dibuja una tarjeta por cada uno. Hoy hay tres: una memoria RAM DDR5 Adata, una tarjeta de video Asus GeForce RTX 3050 y una fuente de poder MSI de 650 W. Cada tarjeta incluye foto, nombre, código ID, disponibilidad, precio normal, oferta por transferencia, otro medio de pago y el botón “Agregar al carrito”.
- ❖ **Footer:** pie de página con los nombres del equipo y el aviso ©2026 LevelUp.

4.2. Páginas del sitio

- ❖ **Inicio:** reúne el bloque de bienvenida (Hero) y el catálogo de productos.
- ❖ **Nosotros:** presenta la misión y la visión de LevelUp junto a una imagen de trabajo en equipo.
- ❖ **Sucursal:** muestra la dirección (Av. Vicuña Mackenna 4917, San Joaquín, Santiago), el horario (lunes a viernes de 9:00 a 18:00, sábado de 10:00 a 16:00 y domingo cerrado), el teléfono, el correo y un mapa de Google Maps.
- ❖ **Contacto:** formulario con tres campos obligatorios: Nombre completo, Correo y Mensaje. La revisión de los datos la hace el propio sitio, como se explica en la sección 7.

4.3. Navegación entre páginas

La navegación usa **react-router**. En el archivo **index.js** se activa el enrutador (BrowserRouter). En **App.js** se definen cuatro rutas: inicio (“/”), “/nosotros”, “/Sucursal” y “/Contacto”. CustomNav y Footer quedan fuera de las rutas, por eso aparecen en todas las páginas y el cliente cambia de sección sin que el sitio se recargue.

5. Propiedades y estado (React)

React usa dos ideas básicas para que el sitio responda a lo que hace el cliente. Las **propiedades (props)** son los datos que un componente recibe desde afuera. El **estado (state)** es la información que un componente guarda y puede cambiar con el tiempo. Cuando el estado cambia, React actualiza la pantalla automáticamente.

5.1. Propiedades (props)

- ❖ **onBuscar:** App le entrega a CustomNav una función. Cuando el cliente busca, CustomNav la usa para avisarle a App qué palabra escribió.
- ❖ **busqueda:** App le entrega la palabra buscada a Home, y Home se la entrega a Catalogo para que filtre los productos.
- ❖ **Propiedades de react-bootstrap:** CustomNav también configura las piezas de la barra con datos, por ejemplo `expand="lg"` (cuándo se pliega el menú), `to="/Nosotros"` (a qué página lleva un enlace) y `title="Más"` (el texto del menú desplegable).

5.2. Estado (state)

- ❖ **busqueda (en App):** guarda la palabra que se está buscando. Está en App porque el buscador vive en CustomNav y los productos en Catalogo, y App es el componente que contiene a ambos.
- ❖ **termino (en CustomNav):** guarda lo que el cliente va escribiendo en el buscador, letra por letra.
- ❖ **formulario, errores y resultado (en Contacto):** guardan lo escrito en los tres campos, los mensajes de error de cada campo y el aviso de envío correcto.

La Figura 3 muestra cómo viajan los datos desde que el cliente escribe en el buscador hasta que ve los productos.

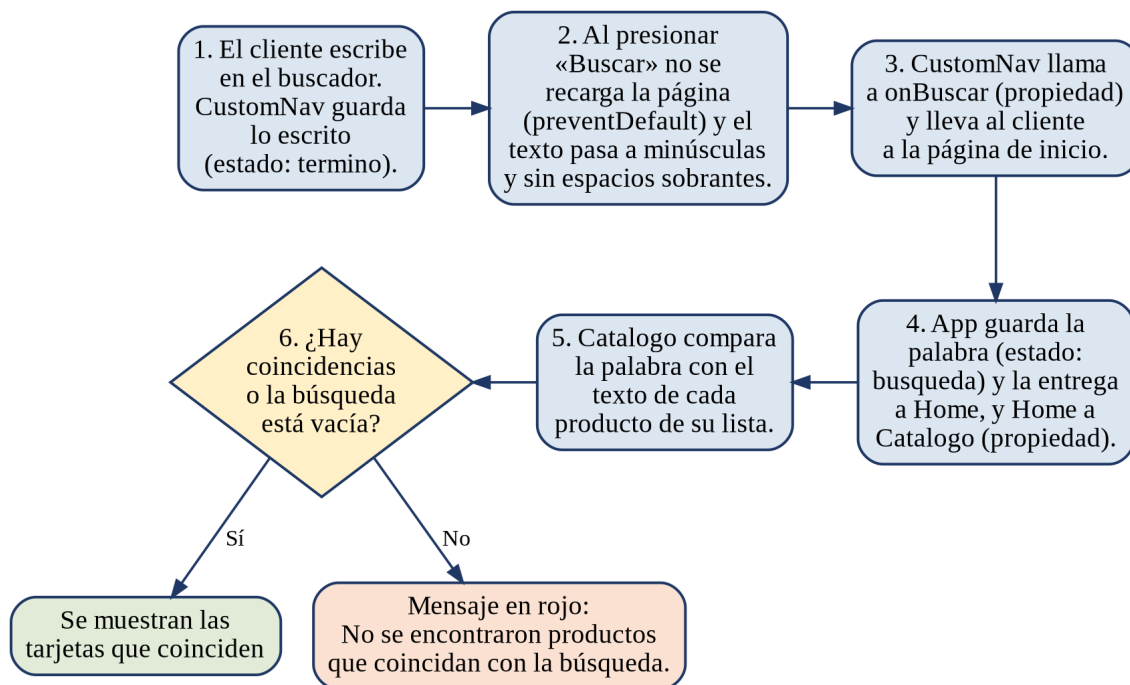


Figura 3. Flujo del buscador de productos con propiedades y estado.

6. Validaciones que se mantuvieron

La versión HTML tenía dos funciones en el archivo funciones.js: el buscador y la validación del formulario. Las dos pasaron a React y mantienen las mismas reglas.

6.1. Buscador de productos

Se mantuvo la lógica original:

- ❖ La palabra se pasa a minúsculas y se quitan los espacios sobrantes, para que “ram”, “RAM” y “ RAM ” den el mismo resultado.
- ❖ La palabra se compara con el texto de cada tarjeta: nombre, código, disponibilidad y precios.
- ❖ Si el buscador está vacío, se muestran todos los productos.
- ❖ Si ningún producto coincide, aparece en rojo el mensaje “No se encontraron productos que coincidan con la búsqueda.”

Lo que cambió es la forma de mostrar el resultado. Antes el sitio ocultaba las tarjetas que no coincidían. Ahora React dibuja solo las que coinciden.

6.2. Formulario de contacto

Se mantuvieron las mismas reglas y los mismos mensajes de la versión HTML:

- ❖ El nombre es obligatorio: “Debe ingresar su nombre.”
- ❖ El correo es obligatorio: “Debe ingresar su correo.”
- ❖ El correo debe tener un formato válido: “Correo inválido. Ejemplo: nombre@correo.cl”
- ❖ El mensaje es obligatorio: “Debe escribir un mensaje.”
- ❖ El mensaje debe tener al menos 10 caracteres: “El mensaje debe tener al menos 10 caracteres.”
- ❖ Si todo está correcto, aparece en verde “Formulario completado correctamente.” y los campos se vacían.

Al migrar se hicieron cuatro mejoras:

- ❖ **Todos los errores a la vez:** antes el sitio mostraba solo el primer error. Ahora muestra el de cada campo que esté mal.
- ❖ **Correo más estricto:** antes bastaba con que tuviera “@” y un punto. Ahora debe tener la forma texto@texto.texto.
- ❖ **Error que desaparece al escribir:** cuando el cliente corrige un campo, su mensaje de error se borra.
- ❖ **Mensajes propios:** se desactivó el aviso automático del navegador (noValidate) para que el cliente vea siempre los mensajes del sitio.

La Figura 4 resume el recorrido de una validación.

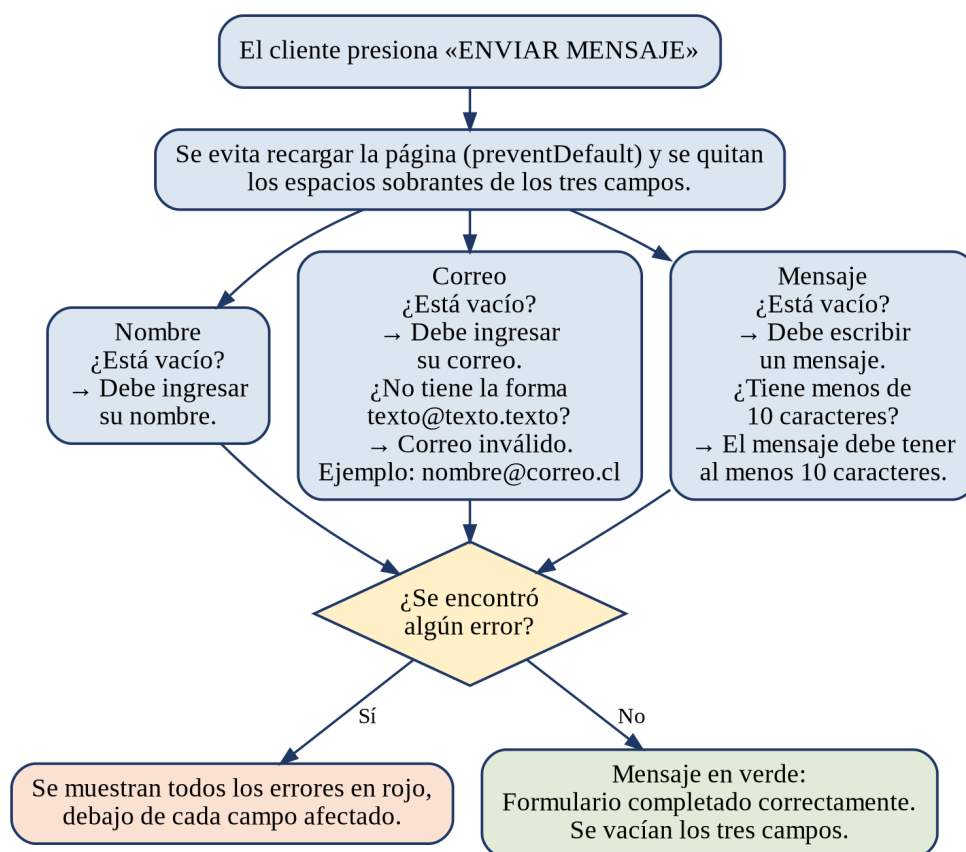


Figura 4. Flujo de validación del formulario de contacto.

FORMULARIO DE CONTACTOS

NOMBRE COMPLETO

Debe ingresar su nombre.

CORREO

Ejemplo: nombre@correo.cl

Debe ingresar su correo.

MENSAJE

Escribe tu consulta aquí

Debe escribir un mensaje.

ENVIAR MENSAJE

Figura 5 Envío vacío con errores debajo de cada campo

FORMULARIO DE CONTACTOS

NOMBRE COMPLETO

CORREO

Ejemplo: nombre@correo.cl

MENSAJE

Escribe tu consulta aquí

Formulario completado correctamente.

ENVIAR MENSAJE

Figura 6 Confirmación local y campos vacíos tras datos válidos

7. Pruebas unitarias (Karma y Jasmine)

Una prueba unitaria revisa, por separado, que una pieza del sitio haga lo que debe hacer. Para esto usamos **Jasmine**, que define los casos de prueba y verifica los resultados, y **Karma**, que ejecuta esas pruebas en un navegador.

7.1. Configuración del entorno

La configuración está en el archivo **karma.conf.js**, dentro de la carpeta proyecto/. Karma busca los archivos terminados en **.test.js** dentro de src/; hoy existe uno, **App.test.js**. Las pruebas corren en Chrome y los resultados se muestran en la consola con el reporte “progress”. Para probar la navegación sin abrir el sitio completo se usa **MemoryRouter**, que simula las rutas de react-router dentro de la prueba. Las pruebas se ejecutan con el comando **npx karma start**. La Figura 5 muestra el recorrido completo.

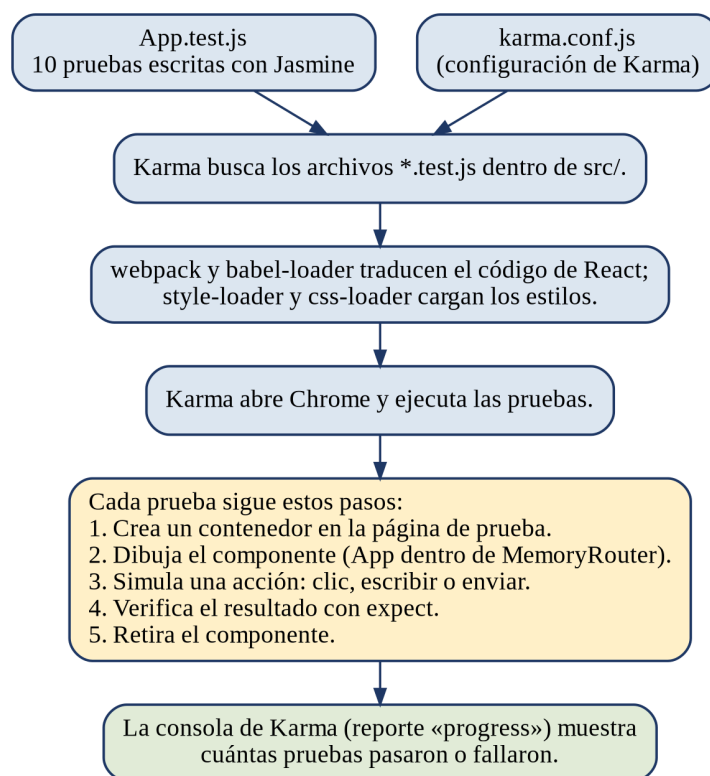


Figura 5. Cómo Karma y Jasmine ejecutan las pruebas.

7.2. Casos de prueba

El archivo App.test.js agrupa 10 pruebas en una suite llamada “Suite de Pruebas Unitarias Frontend (Jasmine + Karma)”. Las pruebas 1 a 5 revisan la aplicación real:

- ❖ **Prueba 1:** la aplicación se dibuja sin errores dentro de MemoryRouter.
- ❖ **Prueba 2:** existe una barra de navegación (nav).
- ❖ **Prueba 3:** La página muestra texto.
- ❖ **Prueba 4:** hay al menos un enlace de navegación.
- ❖ **Prueba 5:** hay elementos con clases CSS, como las de Bootstrap.

Las pruebas 6 a 10 usan componentes pequeños de ejemplo para demostrar las técnicas que el sitio emplea:

- ❖ **Prueba 6:** un contador parte en 0 y sube a 1 al hacer clic (estado con useState).
- ❖ **Prueba 7:** un campo de texto guarda lo que escribe la persona (estado).
- ❖ **Prueba 8:** un espía de Jasmine registra que una función se ejecutó una vez al hacer clic.
- ❖ **Prueba 9:** un componente muestra el título que recibe por propiedades.
- ❖ **Prueba 10:** al enviar un formulario se usa preventDefault para evitar que la página se recargue. Es la misma técnica del buscador y del formulario de contacto.

```
PS C:\Users\sebyv\Downloads\Web-Fullstack\proyecto> npx.cmd karma start karma.conf.js --single-run
Webpack bundling...
asset commons.js 5.34 MiB [emitted] (name: commons) (id hint: commons)
asset c5b461d2dbde6df32319.jpg 183 KiB [emitted] [immutable] [from: src/asset/watermarked_img_7179342786634952262.jpg]
asset runtime.js 21.4 KiB [emitted] (name: runtime)
asset App.test.3634845797.js 402 bytes [emitted] (name: App.test.3634845797)
Entrypoint App.test.3634845797 5.37 MiB (183 KiB) = runtime.js 21.4 KiB commons.js 5.34 MiB App.test.3634845797.js 402 B
webpack 5.111.1 compiled successfully in 1419 ms
07 10 2026 05:16:11.589:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
07 10 2026 05:16:11.590:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
07 10 2026 05:16:11.604:INFO [launcher]: Starting browser Chrome
07 10 2026 05:16:11.961:INFO [Chrome 154.0.0.0 (Windows 10)]: Connected on socket X4xRPLVKdEkRsT7zAAAA with id 50210162
Chrome 154.0.0.0 (Windows 10): Executed 1 of 10 SUCCESS (0 secs / 0.025 secs)
Chrome 154.0.0.0 (Windows 10): Executed 6 of 10 SUCCESS (0 secs / 0.035 secs)
Chrome 154.0.0.0 (Windows 10): Executed 8 of 10 SUCCESS (0 secs / 0.042 secs)
Chrome 154.0.0.0 (Windows 10): Executed 10 of 10 SUCCESS (0.057 secs / 0.044 secs)
TOTAL: 10 SUCCESS
PS C:\Users\sebyv\Downloads\Web-Fullstack\proyecto>
```

7.3. Cobertura

Actualmente, el proyecto cuenta con diez pruebas unitarias definidas en `App.test.js`, ejecutadas mediante Jasmine y Karma en Google Chrome. La ejecución presentada finalizó con el resultado «TOTAL: 10 SUCCESS», lo que indica que todas las comprobaciones incluidas fueron aprobadas.

Las primeras cinco pruebas revisan aspectos generales de la aplicación real: su renderización, la presencia de la barra de navegación, contenido de texto, enlaces y elementos con clases CSS. Las otras cinco utilizan componentes de ejemplo para comprobar técnicas como la actualización de estado, el manejo de eventos, el uso de propiedades, los espías de funciones y la prevención de la recarga de formularios.

Estas pruebas ofrecen una verificación inicial, pero todavía no cubren todas las funcionalidades críticas de LevelUp. Faltan casos que comprueben directamente el filtrado de productos, el mensaje de búsqueda sin coincidencias, la recuperación del catálogo al enviar una búsqueda vacía y las reglas de validación del formulario de contacto.

También es necesario distinguir entre pruebas aprobadas y cobertura de código. Que las diez pruebas pasen no significa que se haya probado el 100 % de la aplicación. Actualmente no se presenta un reporte que mida los porcentajes de líneas, funciones, instrucciones y ramas ejecutadas durante las pruebas. Por ello, la cobertura actual puede describirse como parcial en cuanto a las funcionalidades verificadas, pero su porcentaje permanece sin determinar. Como mejora futura, se propone ampliar las pruebas de los componentes reales y configurar la generación del informe de cobertura para identificar qué partes del código requieren más comprobaciones.

8. Pendientes y mejoras futuras

Quedan cuatro mejoras para una próxima etapa:

- ❖ **Carrito:** el botón “Agregar al carrito” todavía no ejecuta ninguna acción. Se agregará un contador con useState que suba con cada clic.
- ❖ **Tarjeta de producto reutilizable:** hoy las tarjetas se dibujan dentro de Catalogo a partir de una lista. Se separará la tarjeta en su propio componente, que reciba los datos por propiedades.
- ❖ **Más pruebas:** hoy las pruebas sobre el sitio real son 5 y revisan su estructura. Faltan pruebas del buscador y de las reglas del formulario de contacto.
- ❖ **Botón “Explorar Productos”:** todavía no lleva al catálogo.

9. Control de versiones con GitHub

El control de versiones es un sistema que guarda la historia de un proyecto: qué se cambió, cuándo y quién lo hizo. Permite volver a una versión anterior si algo falla y trabajar en equipo sin borrar el trabajo de otros. Para esto usamos Git, que guarda esa historia en cada computador, y GitHub, el sitio web donde se aloja el repositorio.

9.1. El repositorio del proyecto

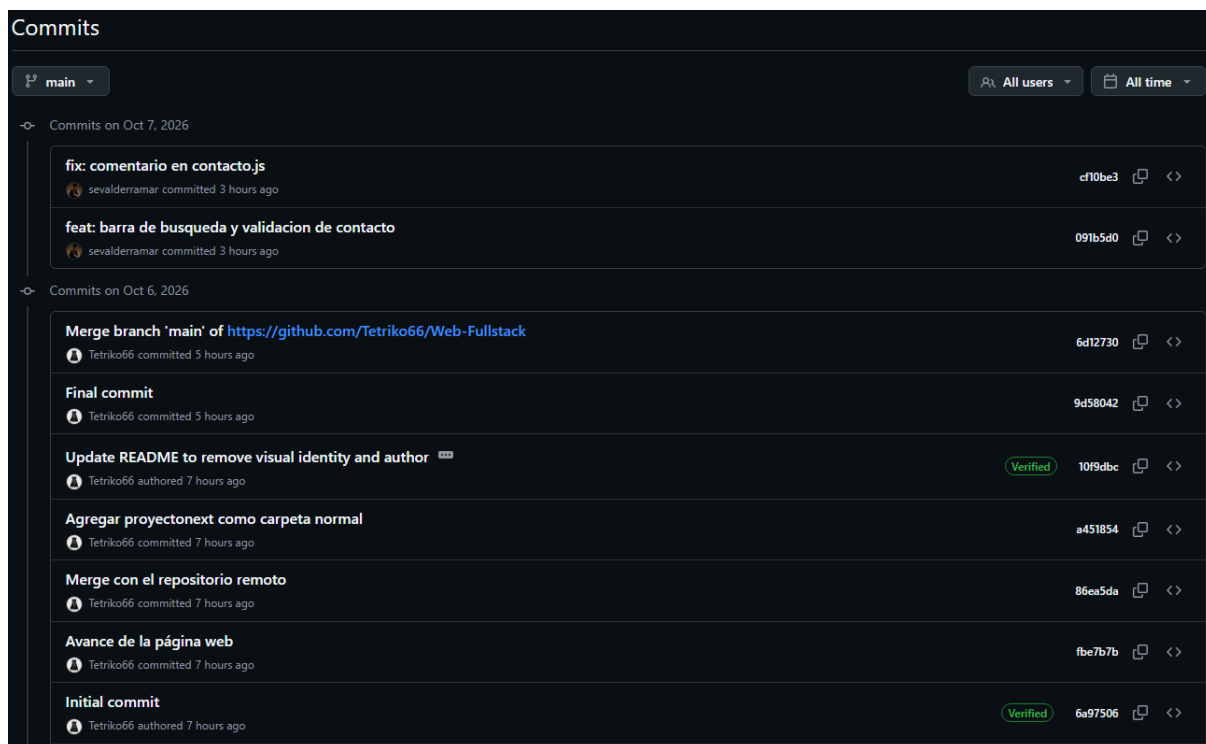
- ❖ El repositorio se llama Web-Fullstack y está en <https://github.com/Tetrik066/Web-Fullstack>.
- ❖ Rama principal: main.
- ❖ Carpeta Paginahtml/: contiene la versión original en HTML, CSS y JavaScript.
- ❖ Carpeta proyecto/: contiene la versión en React, con su configuración de Karma.
- ❖ Archivo README.md: explica de qué trata el proyecto.
- ❖ Archivo .gitignore: evita subir la carpeta node_modules, que pesa mucho y se puede volver a instalar.
- ❖ Mantener las dos versiones en el mismo repositorio permitió conservar la versión original mientras se construía la nueva.

9.2. Commits

Un commit es una "foto" del proyecto en un momento determinado, con un mensaje que explica qué cambió. El equipo escribe mensajes cortos que empiezan con el tipo de cambio. Por ejemplo, "fix: comentario en contacto.js" indica una corrección (fix) en el archivo de contacto.

El flujo de trabajo usa estos comandos:

- ❖ git clone: copia el repositorio al computador.
- ❖ git add: elige los archivos que se van a guardar.
- ❖ git commit -m "mensaje": guarda esos cambios con una descripción.
- ❖ git push: sube los commits a GitHub.
- ❖ git pull: baja los cambios que hicieron los compañeros.



Commits

main

Commits on Oct 7, 2026

- fix: comentario en contacto.js
sevalderramar committed 3 hours ago
cf10be3
- feat: barra de busqueda y validacion de contacto
sevalderramar committed 3 hours ago
091b5d0

Commits on Oct 6, 2026

- Merge branch 'main' of <https://github.com/Tetrik066/Web-Fullstack>
Tetrik066 committed 5 hours ago
6d12730
- Final commit
Tetrik066 committed 5 hours ago
9d58042
- Update README to remove visual identity and author
Tetrik066 authored 7 hours ago
Verified 10f9dbc
- Agregar proyectonext como carpeta normal
Tetrik066 committed 7 hours ago
a451854
- Merge con el repositorio remoto
Tetrik066 committed 7 hours ago
86ea5da
- Avance de la página web
Tetrik066 committed 7 hours ago
fbc7b7b
- Initial commit
Tetrik066 authored 7 hours ago
Verified 6a97506

9.3. Trabajo en equipo

Como el proyecto lo desarrollamos tres personas, el historial de commits permite ver el aporte de cada integrante. Todos trabajamos sobre el mismo repositorio, y GitHub registra cada cambio con su autor y su fecha.

9.4. Beneficios para este proyecto

- ❖ Respaldo: el código queda guardado en la nube y no depende de un solo computador.
- ❖ Historia: se puede revisar cómo avanzó la migración de HTML a React.
- ❖ Seguridad: si un cambio causa un error, se puede volver a una versión anterior.
- ❖ Orden: el README y la separación en carpetas permiten que otra persona entienda y ejecute el proyecto.

10. Conclusión

El proyecto LevelUp pasó de un sitio HTML con código repetido a una aplicación en React construida con componentes reutilizables. El menú (CustomNav), el bloque de bienvenida (Hero), el catálogo (Catalogo) y el pie de página (Footer) se escriben una sola vez y se usan en todo el sitio. Gracias a react-router, el cliente cambia entre las cuatro páginas (Inicio, Nosotros, Sucursal y Contacto) sin que el sitio se recargue, lo que hace la navegación más rápida y fluida.

En cuanto a la funcionalidad, la migración usó propiedades y estado de React. La palabra buscada se guarda en App y viaja por propiedades hasta el catálogo, que muestra sólo los productos que coinciden. El formulario de contacto guarda lo escrito, los errores y el resultado en su propio estado. Las reglas del buscador y del formulario se mantuvieron, y se agregaron mejoras: ahora se muestran todos los errores a la vez, el correo se revisa con más rigor, el error desaparece cuando el cliente corrige el campo y los mensajes son siempre los del sitio.

En el diseño se conservó la identidad visual azul, con Bootstrap y react-bootstrap, y el sitio se adapta a celulares, tablets y computadores. Las tecnologías usadas, como React, react-router, Bootstrap, Node.js y npm, permiten que el sitio sea más fácil de mantener y de ampliar que la versión original.

En calidad y organización, se definieron 10 pruebas con Jasmine y Karma, y se midieron sus resultados y su cobertura. Estas pruebas confirman que la aplicación funciona y muestran las técnicas que el sitio emplea. Además, el uso de Git y GitHub dejó un respaldo del código, un historial de cambios visible para todo el equipo y la posibilidad de conservar la versión HTML original junto a la nueva.